chatcore 插件深度审查报告

> 版本：1.0.39 | 类型：后端插件 | 代码量：~13,500 行 / 71 文件
>
> 审查日期：2026-03-27

综合评分

维度	评分	简评
功能性	8/10	IM 核心功能完整，群聊/离线消息/推送全覆盖
安全性	4/10	多个越权漏洞，缺乏输入校验
架构设计	6/10	分层合理但可扩展性差
接口设计	5/10	REST 规范不统一，缺少版本管理
错误处理	4/10	大量静默失败和不一致的错误响应
可测试性	3/10	无测试代码，强耦合使得测试困难
总评	**5/10**	功能强大的单体，技术债务沉重

一、功能性分析

功能版图（✅ 全面覆盖 IM 核心需求）

模块	功能	完备度
消息	文本、图片、语音、视频、文件、位置、引用回复	✅ 完善
WebSocket	长连接、心跳检测、消息推送	✅ 基本可用
群聊	创建/加入/退出、5 种房间类型、成员角色管理	✅ 功能全面
离线消息	批量投递（100/批）、确认删除	✅ 可用
好友	申请/同意/拒绝/删除、关系状态管理	✅ 完整
推送	6 厂商适配（华为/荣耀/小米/OPPO/VIVO/苹果）	✅ 覆盖广
AI 聊天	DeepSeek 集成、流式响应	✅ 创新
文件上传	图片/头像/文件上传	✅ 基本可用

功能缺陷

缺陷	严重度	说明
单进程架构	🔴 严重	无法水平扩展，WebSocket 连接上限受限于单机
无消息持久化确认	🟠 高	写数据库成功前消息已推送，crash 可导致消息丢失
无消息加密	🟠 高	端到端明文传输和存储
离线消息上限缺失	🟡 中	长期离线用户可能积累数十万条离线消息
无消息撤回	🟡 中	不支持已发送消息的撤回操作
无已读回执	🟡 中	无法确认对方已读

二、安全性批判

🔴 致命问题 1：内存分布式锁在分布式环境无效

```
class OfflineMessageLock {
  static final Map<int, bool> _locks = {};

  static Future<bool> acquire(int userId) async {
    if (_locks[userId] == true) return false;
    _locks[userId] = true;
    return true;
  }

  static void release(int userId) {
    _locks.remove(userId);
  }
}
```

批判：

- 内存 Map 做锁 = 单进程内有效，多实例部署无效
- 同一用户两个实例同时投递离线消息 → 重复投递
- 服务崩溃 → 锁永远不释放（无 TTL 机制）
- 命名为 "分布式锁" 但实际是进程级互斥

改良方案：使用 Redis SETNX 或 MySQL `GET_LOCK()` 实现真正的分布式锁。

🔴 致命问题 2：群管理缺少权限校验

```
// 群主踢人 — 但没检查操作者是否是群主
Future<Response> removeMember(Request request) async {
  final data = await request.readAsString();
  final json = jsonDecode(data);
  final roomId = json['roomId'];
  final userId = json['userId'];

  await _roomRepository.removeMember(roomId, userId);
  return Response.ok(jsonEncode({'success': true}));
}
```

批判：

- 任何用户都可以调用此接口踢掉其他人
- 没验证操作者身份，也没验证操作者是否有管理员权限
- 群房间的 owner/admin 角色形同虚设

🔴 致命问题 3：WebSocket 无攻击防护

```
void handleWebSocket(WebSocket ws, int userId) {
  // 直接接受连接，无速率限制
  _connections[userId] = ws;
```

```
  ws.listen((message) {
    // 解析消息并广播
    _handleMessage(userId, message);
  });
}
```

- 无连接速率限制：恶意客户端可不断建立/断开连接
- 无消息速率限制：可发送消息轰炸
- 无消息大小限制：超大消息可导致 OOM
- 无身份重新验证：令牌过期后连接继续有效

三、架构设计批判

分层架构（合理但不严格）

```
Routes → Middleware → Services → Repositories → Database
  ↓                       ↓
Models                Push Providers
```

优点

✅ **Repository 模式**：数据访问层抽象良好

✅ **DI 容器**：服务依赖通过 DI 注入

✅ **Strategy 模式**：推送提供者可插拔

✅ **中间件链**：认证、日志、CORS 中间件有序组织

缺陷

问题 1：上帝服务 — *ChatService*

```
class ChatService {
  // ~1,500 行的巨型服务

  Future<void> sendMessage(...) { ... }
  Future<void> sendGroupMessage(...) { ... }
  Future<void> handleOfflineMessages(...) { ... }
  Future<void> createRoom(...) { ... }
  Future<void> joinRoom(...) { ... }
  Future<void> leaveRoom(...) { ... }
  Future<void> addFriend(...) { ... }
  Future<void> removeFriend(...) { ... }
  Future<void> uploadFile(...) { ... }
  Future<void> sendPush(...) { ... }
  // ... 还有 30+ 个方法
}
```

批判：

- 一个类承担了消息、群组、好友、文件、推送五个领域的职责
- 1,500+ 行代码，任何修改都可能影响其他功能
- 违反 SRP（单一职责原则），是典型的 God Object

改良方案：

```
class MessageService { ... }       // 消息发送/接收
class RoomService { ... }          // 群聊管理
class FriendService { ... }        // 好友关系
class FileService { ... }          // 文件上传/下载
class NotificationService { ... } // 推送通知
```

问题 2：*WebSocket* 非 *JSON* 标准化

```
void _sendToUser(int userId, Map<String, dynamic> data) {
  final ws = _connections[userId];
  if (ws != null) {
    ws.add(jsonEncode(data));  // JSON 协议
  }
}
```

- 既然 authcore 支持 Protobuf，为何 WebSocket 全用 JSON?
- JSON 编解码开销远大于 Protobuf，高频消息场景下性能损失明显
- 消息格式无版本号，协议升级困难

问题 3：无事件驱动架构

```
Future<void> sendMessage(/* ... */) async {
  // 1. 保存消息到数据库
  await _messageRepo.save(message);
  // 2. 通过 WebSocket 推送
  _sendToUser(receiverId, message);
  // 3. 如果不在线，存离线消息
  if (!_isOnline(receiverId)) {
    await _offlineMessageRepo.save(message);
  }
  // 4. 发送推送通知
  await _pushService.send(receiverId, message);
  // 5. 更新会话列表
  await _conversationRepo.update(message);
}
```

批判：

- 所有操作串行执行在同一个方法中
- 步骤 2 失败 → 步骤 3、4、5 不执行
- 步骤 4 推送耗时可能很长，阻塞主流程
- 无法重试单个步骤

改良方案：引入事件总线，消息保存后发布事件，各模块异步消费：

```
Future<void> sendMessage(/* ... */) async {
```

```
  final savedMessage = await _messageRepo.save(message);
  _eventBus.publish(MessageSavedEvent(savedMessage));
}

// 各处理器独立订阅
class WebSocketDeliveryHandler { ... }  // 订阅 MessageSavedEvent
class OfflineMessageHandler { ... }     // 订阅 MessageSavedEvent
class PushNotificationHandler { ... }   // 订阅 MessageSavedEvent
```

问题 4：连接管理单点

```
static final Map<int, WebSocket> _connections = {};
```

- 内存中的 Map 只属于当前进程
- 部署两个实例 → 用户 A 连实例 1，用户 B 连实例 2 → A 给 B 发消息 → 实例 1 在自己的 `_connections` 中找不到 B → 消息丢失
- 必须引入 Redis Pub/Sub 或消息队列实现跨实例消息路由

四、接口设计批判

问题 1：REST API 不规范

```
// 路由定义（部分示例）
router.post('/api/message/send', chatController.sendMessage);
router.post('/api/room/create', chatController.createRoom);
router.post('/api/room/join', chatController.joinRoom);
router.get('/api/room/members', chatController.getMembers);
router.post('/api/friend/add', chatController.addFriend);
router.post('/api/friend/delete', chatController.deleteFriend);
```

批判：

- `POST /api/friend/delete` → 应为 `DELETE /api/friends/:id`
- `POST /api/room/join` → 应为 `POST /api/rooms/:id/members`
- `POST /api/message/send` → 应为 `POST /api/messages`
- 混用单数/复数：`/room/` vs `members`
- 无 API 版本号：`/api/v1/...`
- 查询操作用 GET，修改操作用 POST/PUT/DELETE

改良方案：

```
// RESTful 规范
router.post('/api/v1/messages', chatController.sendMessage);
router.post('/api/v1/rooms', chatController.createRoom);
router.post('/api/v1/rooms/:roomId/members', chatController.joinRoom);
router.get('/api/v1/rooms/:roomId/members', chatController.getMembers);
router.delete('/api/v1/rooms/:roomId/members/:userId',
chatController.removeMember);
router.post('/api/v1/friends/requests', chatController.addFriend);
```

```
router.delete('/api/v1/friends/:friendId', chatController.deleteFriend);
```

问题 2：响应格式不一致

```
// 成功响应格式 1
return Response.ok(jsonEncode({'success': true, 'data': result}));

// 成功响应格式 2
return Response.ok(jsonEncode({'code': 0, 'message': 'ok', 'data':
result}));

// 成功响应格式 3
return Response.ok(jsonEncode(result));  // 直接返回数据

// 错误响应
return Response(400, body: jsonEncode({'error': 'message'}));
return Response(500, body: 'Internal error');  // 纯字符串
```

批判：

- 三种不同的成功响应格式，前端需要三套解析逻辑
- 错误响应有时是 JSON 有时是纯字符串
- 没有统一的响应包装器

改良方案：

```
class ApiResponse {
  static Response success(dynamic data) => Response.ok(
    jsonEncode({'code': 0, 'data': data}),
    headers: {'content-type': 'application/json'},
  );

  static Response error(int statusCode, String message) => Response(
    statusCode,
    body: jsonEncode({'code': statusCode, 'message': message}),
    headers: {'content-type': 'application/json'},
  );
}
```

问题 3：分页查询参数不统一

```
// 有的接口用 page/limit
router.get('/api/messages?page=1&limit=20');

// 有的接口用 offset/count
router.get('/api/offline-messages?offset=0&count=100');

// 有的不分页
router.get('/api/rooms');  // 返回所有数据
```

五、代码质量

问题 1：DeepSeek AI 集成硬编码

```
class DeepSeekService {
  static const _apiKey = 'sk-xxxxx';  // API Key 硬编码
  static const _model = 'deepseek-chat';
  static const _baseUrl = 'https://api.deepseek.com';

  Future<String> chat(String prompt) async {
    // ...
  }
}
```

API Key 硬编码在源码中，推送到 Git 仓库 = 密钥泄露。

问题 2：文件上传无安全限制

```
Future<Response> uploadFile(Request request) async {
  final bytes = await request.read().expand((b) => b).toList();
  final filename = request.headers['filename'] ?? 'unknown';
  final file = File('uploads/$filename');
  await file.writeAsBytes(bytes);
  return Response.ok(jsonEncode({'url': '/uploads/$filename'}));
}
```

批判：

- 无文件大小限制：攻击者可上传 10GB 文件撑爆磁盘
- 无文件类型校验：可上传 `.exe` 、 `.sh` 等可执行文件
- 路径遍历漏洞： `filename=../../bin/server.dart` 可覆盖任意文件
- 无防病毒扫描

问题 3：消息内容无过滤

```
// 消息直接存储和转发，无任何内容过滤
await _messageRepo.save({
  'content': userInput,  // ← 未转义、未过滤
  'type': 'text',
});
```

XSS 攻击向量：消息内容包含 `<script>` 标签，前端渲染时执行恶意代码。

六、改良建议总结

优先级	改良项	预估工作量
P0	文件上传安全：大小限制 + 类型校验 + 路径过滤	3h
P0	API Key 移入环境变量	0.5h
P0	群管理权限校验	4h
P0	WebSocket 防攻击（速率限制 + 消息大小限制）	4h
P1	拆分 ChatService 为 5 个独立服务	16h
P1	统一响应格式	4h
P1	统一 REST API 规范	8h
P1	内存锁替换为 Redis 分布式锁	4h
P2	消息内容 XSS 过滤	2h
P2	引入事件驱动架构	16h
P2	WebSocket 支持 Protobuf 协议	8h
P3	支持消息撤回 + 已读回执	8h
P3	连接管理支持多实例	8h